

Отчёт по лабораторной работе №6

по дисциплине «Архитектура компьютера»

Дембирел Тимур Леонидович

Российский университет дружбы народов
Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей
Москва 2025г.

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
3	Ответы на вопросы	15
4	Задание для самостоятельной работы	17
5	Вывод	19

Список иллюстраций

2.1	Создаем каталог и файл	8
2.2	Вводим текст программы	8
2.3	Создаем и запускаем	9
2.4	Изменяем и проверяем	9
2.5	Создаем и вводим	10
2.6	Создаем и запускаем	10
2.7	Изменяем текст программы	11
2.8	Создаем и запускаем	11
2.9	Заменяем и проверяем	11
2.10	Создаем и вводим	12
2.11	Создаем и проверяем	13
2.12	Изменяем	13
2.13	Создаем и проверяем	13
2.14	Создаем и вводим	14
2.15	Проверяем	14
4.1	Создаем новый файл и пишем текст программы	18
4.2	Проверяем	18

Список таблиц

2.1	Основные команды, используемые при работе	7
-----	---	---

1 Цель работы

Освоение арифметических инструкций языка ассемблера NASM.

2 Выполнение лабораторной работы

В Таблица 2.1 приведены основные команды, используемые при работе с git, элементами программирования и компиляцией программ.

Таблица 2.1: Основные команды, используемые при работе

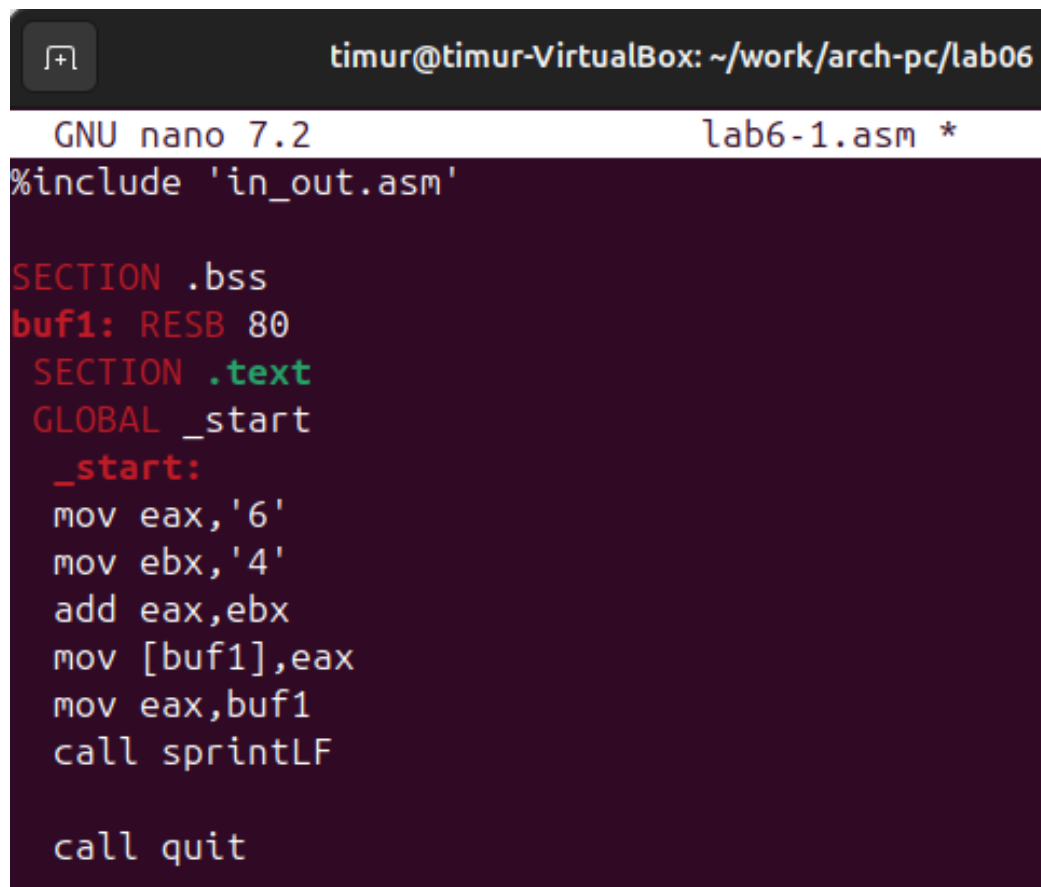
Команды	Описание
<code>mov</code>	Дублирует данные источника в приемнике
<code>add</code>	Целочисленное сложение
<code>touch</code>	Создание файла
<code>nasm</code>	Компилятор ассемблера NASM
<code>git add</code>	Добавление изменений в индекс
<code>git commit</code>	Создание коммита
<code>git push</code>	Отправка изменений на удаленный репозиторий
<code>ld</code>	Связывает объектные файлы в исполняемую программу
<code>xor</code>	Выполняет операцию исключающего ИЛИ
<code>div</code>	Беззнаковое деление
<code>mul</code>	Умножение
<code>call</code>	Вызов программы

Создадим каталог для программ лабораторной работы № 6, перейдем в него и создадим файл „lab6-1.asm“ рис. 2.1.

```
timur@timur-VirtualBox:~$ mkdir ~/work/arch-pc/lab06
timur@timur-VirtualBox:~$ cd ~/work/arch-pc/lab06
timur@timur-VirtualBox:~/work/arch-pc/lab06$ touch lab6-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

Рисунок 2.1: Создаем каталог и файл

Рассмотрим примеры программ вывода символьных и численных значений. Программы будут выводить значения записанные в регистр „eax“. Введем в файл „lab6-1.asm“ текст программы из листинга 6.1 рис. 2.2.



```
timur@timur-VirtualBox: ~/work/arch-pc/lab06
GNU nano 7.2 lab6-1.asm *
#include 'in_out.asm'

SECTION .bss
buf1: RESB 80
SECTION .text
GLOBAL _start
_start:
    mov eax,'6'
    mov ebx,'4'
    add eax,ebx
    mov [buf1],eax
    mov eax,buf1
    call sprintLF

    call quit
```

Рисунок 2.2: Вводим текст программы

Создадим исполняемый файл и запустим его. В результате должно вывести символ „j“, вместо числа 10 рис. 2.3.


```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1
lab6-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./lab6-1
j
timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

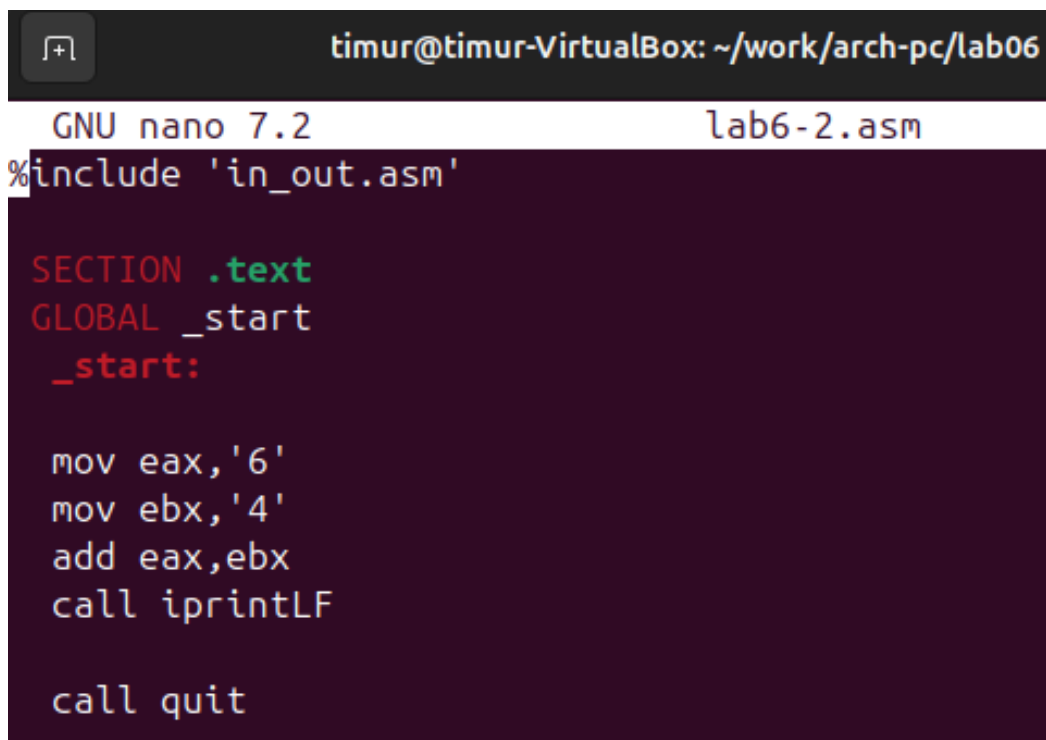
Рисунок 2.3: Создаем и запускаем

Далее изменим текст программы и вместо символов, запишем в регистры числа. Исправим текст программы, создадим исполняемый файл и запустим его. В результате должен вывестись символ с кодом 10, но отображаться он не будет рис. 2.4.

```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1
lab6-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./lab6-1
```

Рисунок 2.4: Изменяем и проверяем

Создадим файл „lab6-2.asm“ в каталоге „~/work/arch-pc/lab06“ и введем в него текст программы из листинга 6.2 рис. 2.5.

A terminal window with a dark background. The title bar shows 'timur@timur-VirtualBox: ~/work/arch-pc/lab06'. The terminal content shows the GNU nano 7.2 editor editing 'lab6-2.asm'. The code includes 'in_out.asm', defines a text section, and contains assembly instructions to calculate 106 and print it.

```
timur@timur-VirtualBox: ~/work/arch-pc/lab06
GNU nano 7.2 lab6-2.asm
%include 'in_out.asm'

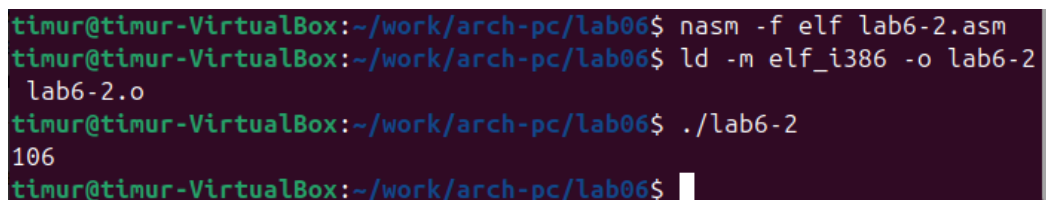
SECTION .text
GLOBAL _start
_start:

    mov eax,'6'
    mov ebx,'4'
    add eax,ebx
    call iprintLF

    call quit
```

Рисунок 2.5: Создаем и вводим

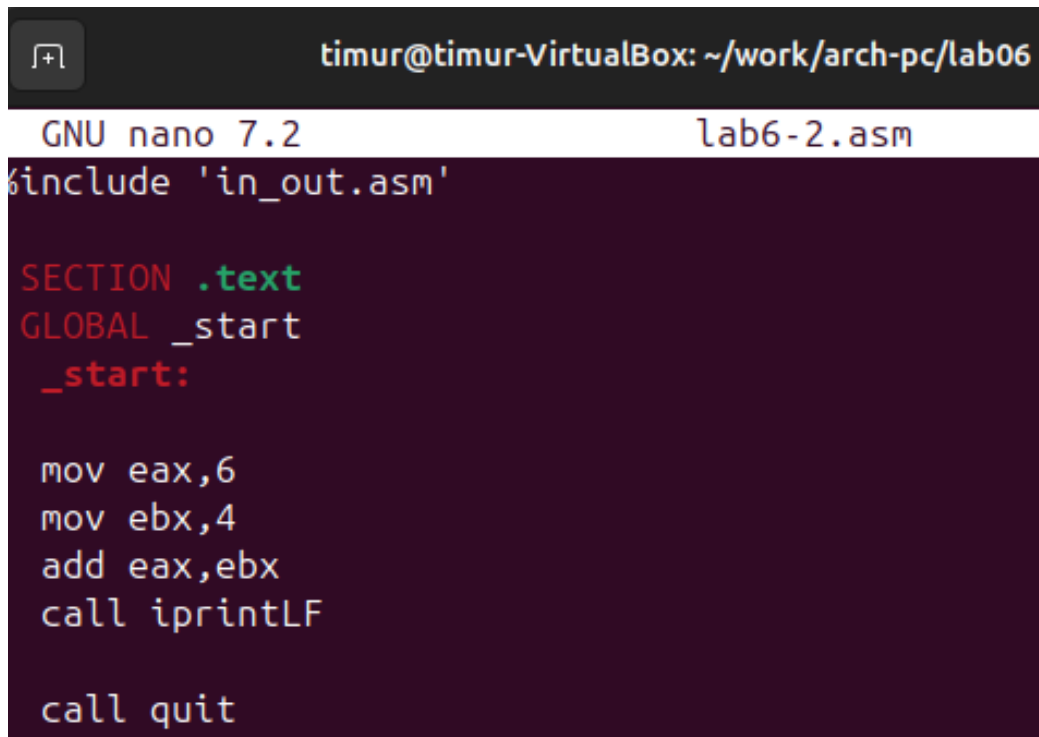
Создайте исполняемый файл и запустите его. В результате работы программы мы получим число 106. Благодаря функции „iprintLF“ мы выводим число, а не символ, кодом которого является это число рис. 2.6.

A terminal window showing the compilation and execution of the assembly program. The user runs 'nasm' to create an object file, 'ld' to create an executable, and then runs the executable, which outputs '106'.

```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./lab6-2
106
timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

Рисунок 2.6: Создаем и запускаем

Аналогично предыдущему примеру изменим символы на числа рис. 2.7.



```
timur@timur-VirtualBox: ~/work/arch-pc/lab06
GNU nano 7.2 lab6-2.asm
#include 'in_out.asm'

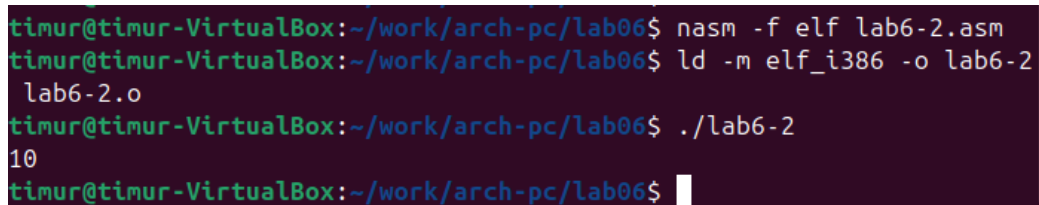
SECTION .text
GLOBAL _start
_start:

mov eax,6
mov ebx,4
add eax,ebx
call iprintLF

call quit
```

Рисунок 2.7: Изменяем текст программы

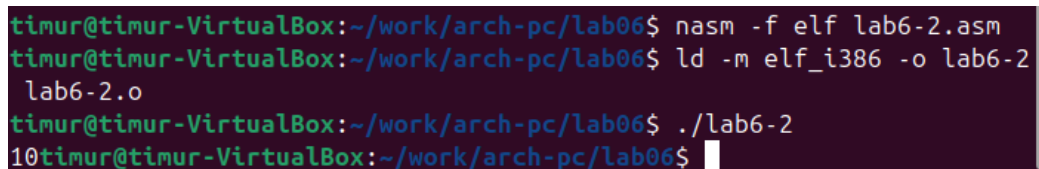
Создадим исполняемый файл и запустим его рис. 2.8.



```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2
lab6-2.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./lab6-2
10
timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

Рисунок 2.8: Создаем и запускаем

Также нас попросили заменить „iprintLF“ на „iprint“, создать исполняемый файл, запустить и найти отличия в выводе функций рис. 2.9.

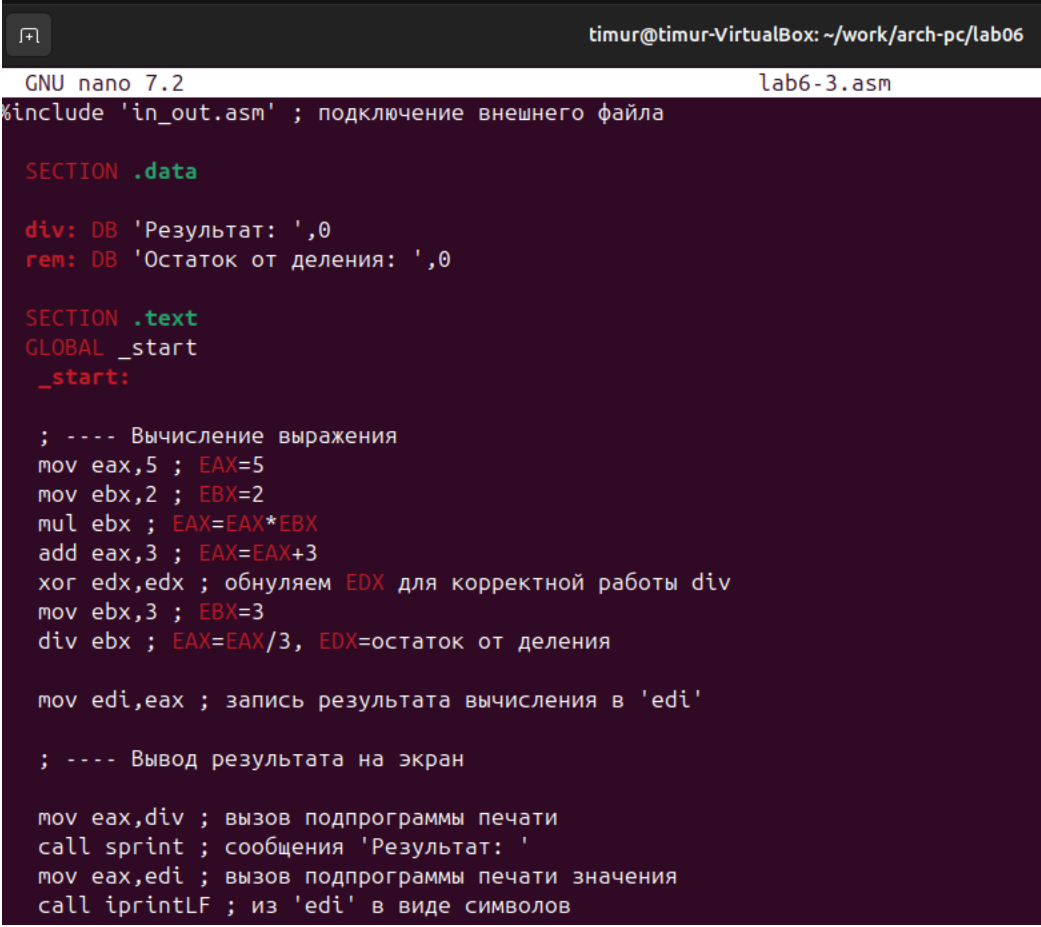


```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2
lab6-2.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./lab6-2
10timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

Рисунок 2.9: Заменяем и проверяем

Отличия в том, что при „ipringLF“ вывод происходит на след строке, а при „iprint“ в начале командной строки.

Чтобы выполнить пример арифметических операций в „NASM“ приведем программу вычисления арифметического выражения $f(x)=(5*2+3)/3$. Создаем файл „lab6-3.asm“ в каталоге „~/work/arch-pc/lab06“ и вводим текст программы из листинга 6.3 рис. 2.10



```
timur@timur-VirtualBox: ~/work/arch-pc/lab06
GNU nano 7.2 lab6-3.asm
%include 'in_out.asm' ; подключение внешнего файла

SECTION .data
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start
_start:

; ---- Вычисление выражения
mov eax,5 ; EAX=5
mov ebx,2 ; EBX=2
mul ebx ; EAX=EAX*EBX
add eax,3 ; EAX=EAX+3
xor edx,edx ; обнуляем EDX для корректной работы div
mov ebx,3 ; EBX=3
div ebx ; EAX=EAX/3, EDX=остаток от деления

mov edi,eax ; запись результата вычисления в 'edi'

; ---- Вывод результата на экран

mov eax,div ; вызов подпрограммы печати
call sprint ; сообщения 'Результат: '
mov eax,edi ; вызов подпрограммы печати значения
call iprintLF ; из 'edi' в виде символов
```

Рисунок 2.10: Создаем и вводим

Создадим исполняемый файл и проверим, что в ответе получится 4 и 1 рис. 2.11.

```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1
timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

Рисунок 2.11: Создаем и проверяем

Далее изменим текст программы для вычисления выражения $f(x)=(4*6+2)/5$ рис. 2.12.

```
; ---- Вычисление выражения
mov eax,4 ; EAX=4
mov ebx,6 ; EBX=6
mul ebx ; EAX=EAX*EBX
add eax,2 ; EAX=EAX+3
xor edx,edx ; обнуляем EDX для корректной работы div
mov ebx,5 ; EBX=5
div ebx ; EAX=EAX/5, EDX=остаток от деления
```

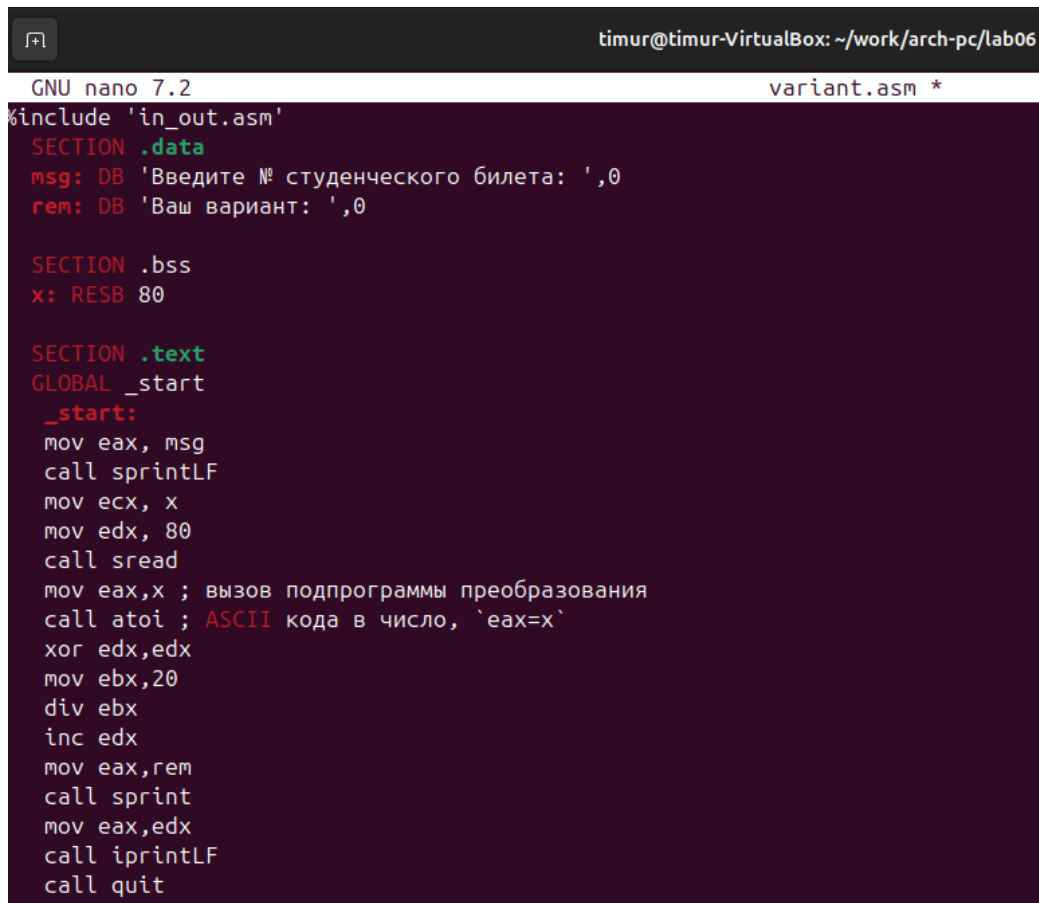
Рисунок 2.12: Изменяем

Создадим исполняемый файл и проверим его работу рис. 2.13.

```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./lab6-3
Результат: 5
Остаток от деления: 1
timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

Рисунок 2.13: Создаем и проверяем

В качестве другого примера рассмотрим программу вычисления варианта задания по номеру студенческого билета. Создадим файл „variant.asm“ в каталоге „~/work/arch-pc/lab06“ и введем туда текст программы из листинга 6.4 рис. 2.14.



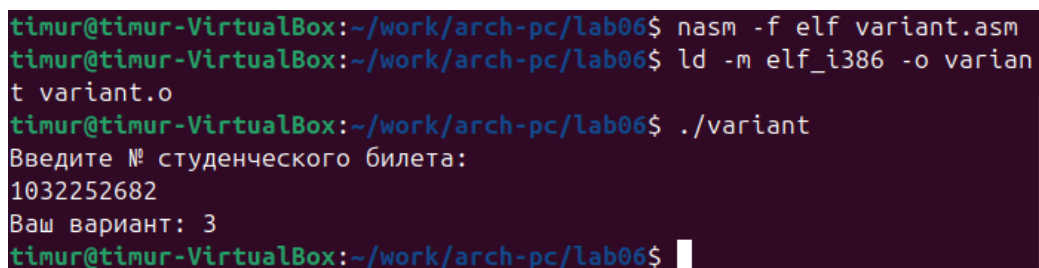
```
timur@timur-VirtualBox: ~/work/arch-pc/lab06
GNU nano 7.2 variant.asm *
#include 'in_out.asm'
SECTION .data
msg: DB 'Введите № студенческого билета: ',0
rem: DB 'Ваш вариант: ',0

SECTION .bss
x: RESB 80

SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprintLF
mov ecx, x
mov edx, 80
call sread
mov eax, x ; вызов подпрограммы преобразования
call atoi ; ASCII кода в число, `eax=x`
xor edx, edx
mov ebx, 20
div ebx
inc edx
mov eax, rem
call sprint
mov eax, edx
call iprintLF
call quit
```

Рисунок 2.14: Создаем и вводим

Создаем исполняемый файл и проверяем его рис. 2.15.



```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf variant.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./variant
Введите № студенческого билета:
1032252682
Ваш вариант: 3
timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

Рисунок 2.15: Проверяем

3 Ответы на вопросы

1. Какие строки листинга 6.4 отвечают за вывод на экран сообщения „Ваш вариант:“?

Ответ: „mov eax,rem“, „call sprint“

2. Для чего используются следующие инструкции? „mov ecx, x“ „mov edx, 80“ „call sread“

Ответ: Эти инструкции используются для ввода данных от пользователя

3. Для чего используется инструкция «call atoi»

Ответ: Инструкция „call atoi“ вызывает подпрограмму преобразования ASCII-строки в число. Она преобразует введенную строку (номер студенческого билета) в целое число, которое сохраняется в регистре eax

4. Какие строки листинга 6.4 отвечают за вычисления варианта?

Ответ: „xor edx,edx“ „mov ebx,20“ „div ebx“ „inc edx“

5. В какой регистр записывается остаток от деления при выполнении инструкции «div ebx»?

Ответ: Остаток от деления записывается в регистр „edx“

6. Для чего используется инструкция «inc edx»?

Ответ: Инструкция „inc edx“ увеличивает значение в регистре „edx“ на 1. Поскольку остаток от деления на 20 может быть от 0 до 19, а варианты обычно нумеруются с 1 до 20, эта инструкция преобразует остаток в диапазон 1-20

7. Какие строки листинга 6.4 отвечают за вывод на экран результата вычислений?

Ответ: „mov eax,edx“ „call iprintLF“

4 Задание для самостоятельной работы

1. Написать программу вычисления выражения $y=f(x)$. Программа должна выводить выражение для вычисления, выводить запрос на ввод значения x , вычислять заданное выражение в зависимости от введенного x , выводить результат вычислений. Вид функции $f(x)$ выбрать из таблицы 6.3 вариантов заданий в соответствии с номером полученным при выполнении лабораторной работы. Создайте исполняемый файл и проверьте его работу для значений x_1 и x_2 из 6.3 рис. 4.1.

```
timur@timur-VirtualBox: ~/work/arch-pc/lab06
GNU nano 7.2 ind-work.asm *
#include 'in_out.asm'

SECTION .data
    msg: DB 'Введите значение x: ',0
    res: DB 'Результат: y = (2 + x)^2 = ',0

SECTION .bss
    x: RESB 80

SECTION .text
GLOBAL _start
_start:
    mov eax, msg
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi
    add eax, 2      ; EAX = 2 + x
    mov ebx, eax    ; Сохраняем (2 + x) в EBX
    mul ebx         ; EAX = (2 + x) * (2 + x)
    mov edi, eax
    mov eax, res
    call sprint
    mov eax, edi
    call iprintLF
    call quit
```

Рисунок 4.1: Создаем новый файл и пишем текст программы

Создаем исполняемый файл и проверяем рис. 4.2.

```
timur@timur-VirtualBox:~/work/arch-pc/lab06$ nasm -f elf ind-work.asm
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant ind-work.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ld -m elf_i386 -o ind-work ind-work.o
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./ind-work
Введите значение x: 2
Результат: y = (2 + x)^2 = 16
timur@timur-VirtualBox:~/work/arch-pc/lab06$ ./ind-work
Введите значение x: 8
Результат: y = (2 + x)^2 = 100
timur@timur-VirtualBox:~/work/arch-pc/lab06$
```

Рисунок 4.2: Проверяем

5 Вывод

В ходе данной лабораторной работы я освоил арифметические инструкции языка ассемблера „NASM“.